

The conditional 4D base diffusion model — architecture & conditioning explained

1. What the model is, in one paragraph
2. The diffusion backbone (EDM)
3. The network: a factorized space-time U-Net
4. Conditioning pathway 1: location as per-pixel coordinate channels
5. Conditioning pathway 2: time as cyclic harmonics (the deep dive)
6. Sampling (how a conditional block is generated)
7. References

The conditional 4D base diffusion model — architecture & conditioning explained

Companion to `docs/conditional-base-changes.md` (the decision log). That doc records **what** was decided and by whom; this one explains **how the model works** and where each idea comes from. Code: `src/eval/windeval/generators/infinite_diffusion/spacetime.py` (model), `data.py` (dataset + conditioning features), `train.py` (loop).

1. What the model is, in one paragraph

It is an **EDM diffusion model** (Karras et al., 2022 [1]) that learns the distribution of short **space-time blocks of wind fields, conditioned on where and when they occur**:

$p(4 \text{ consecutive hourly wind frames over a } 64 \times 64 \text{ crop, all 18 vertical levels} \mid \text{the crop's latitude/longitude, the date, and the hour})$

Formally the sample is a tensor $\mathbf{x} \in \mathbb{R}^{\tau \times \mathbf{C} \times \mathbf{H} \times \mathbf{W}}$ with $\tau = 4$ frames, $\mathbf{C} = 36$ channels (18 model levels $\times$ two wind components u, v , interleaved), $\mathbf{H} = \mathbf{W} = 64$ pixels at 0.25° ($\approx 22\text{--}28$ km/pixel). This is the “4D” object: two horizontal dimensions, the vertical carried as channels, and time as the block axis. Trained well, it becomes the window denoiser Φ that the InfiniteDiffusion machinery tiles into unbounded, seamless, lazily-evaluated wind fields — but the base model is trained and judged on single blocks first (the project’s standing model-vs-machinery separation).

Why *conditional*? The previous model was unconditional: it learned one pooled “climate soup” over all seasons and places it saw. Its documented failure on the benchmark was under-dispersion — too few extreme winds (tail error $\sim 5\times$ the sampling floor). Extremes are creatures of *regime*: a January jet over the NE Pacific produces winds a July anticyclone never will. A model that knows the date and place can allocate its probability mass per-regime instead of averaging across them, which is the causal story behind the tail gap. Conditioning is also what the RL environment needs operationally: “give me winds near San Francisco on an April morning” is a conditional query.

2. The diffusion backbone (EDM)

Diffusion models learn to reverse a noising process. EDM [1] formulates this cleanly: train a **denoiser** $\mathbf{D}(\mathbf{x}_{\text{noisy}}; \sigma, \mathbf{c})$ that, given a field corrupted with Gaussian noise of known scale σ (and conditioning $\mathbf{c}$), predicts the clean field. Sampling then runs

an ODE from pure noise ($\sigma_{\max} = 80$) down to $\sigma \approx 0$, calling the denoiser at each of ~ 18 noise levels (we use EDM’s 2nd-order Heun scheme — two denoiser calls per step, giving second-order accuracy in σ).

Two EDM details matter for reading our code (`EDMPrecondSpaceTime`):

- **Preconditioning.** The raw network F is wrapped as $D(x; \sigma) = c_skip(\sigma) \cdot x + c_out(\sigma) \cdot F(c_in(\sigma) \cdot x; c_noise(\sigma))$, where the c -coefficients are fixed functions of σ (Karras Eq. 7). This keeps the network’s inputs and training targets at unit scale across ten orders of magnitude of σ — a major reason EDM trains stably. Our per-(level, variable) normalization of the wind data (each level’s u and v standardized separately, because wind variance swings $\sim 10\times$ with altitude) makes EDM’s `sigma_data = 1` assumption hold.
- **Training objective.** Sample a clean block x_0 , a noise scale σ from a lognormal, form $x_0 + \sigma \epsilon$, and regress D back to x_0 with EDM’s σ -dependent loss weight. One σ per block, shared by all τ frames — the block is denoised *jointly*, which is what lets the model learn temporal coherence (a video-diffusion idea [2]).

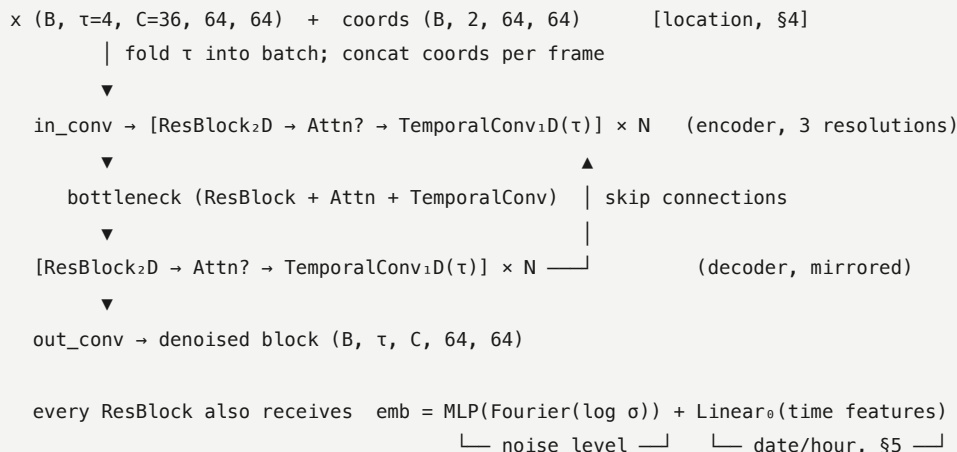
3. The network: a factorized space-time U-Net

SpaceTimeUNet is a standard convolutional U-Net (three resolutions, channel widths $128 \rightarrow 256 \rightarrow 256 \rightarrow 256$, GroupNorm + SiLU ResBlocks, self-attention at the two coarsest resolutions) with one structural idea worth understanding: **space and time are handled by separate, factorized operators rather than 3D convolutions.**

- The proven 2D spatial ResBlocks run **per-frame**: the τ axis is folded into the batch ($B \cdot \tau$ images), so each frame is processed by exactly the machinery the static spatial model validated.
- After every spatial block, a **TemporalConv** — a 1D convolution along τ at each spatial location and channel — mixes information *between* frames.

Why factorize? The three axes are wildly anisotropic: ~ 28 km per horizontal pixel, ~ 380 m per vertical level (which is why levels are channels, not a conv axis), 1 hour per frame. An isotropic 3D kernel would pretend these are comparable. Factorized spatial+temporal operators are the standard resolution of this problem in video models — $R(2+1)D$ decompositions for video CNNs (Tran et al., 2018 [3]) and the spatial-layers-then-temporal-layers pattern of video diffusion models [2, 4].

The detail that makes it safe: **every TemporalConv is zero-initialized** as a residual ($x + \text{Conv1d}(x)$ with the conv’s weights starting at exactly zero). An untrained temporal path therefore computes the identity — the freshly initialized space-time model is the per-frame static model, and temporal coupling is learned strictly on top. This “start-as-identity, learn the delta” initialization is the same trick that makes ControlNet [5] and AnimateDiff’s temporal modules [6] trainable without wrecking their pretrained backbones. We use it twice (see §5: the time-conditioning pathway is also zero-init).



4. Conditioning pathway 1: location as per-pixel coordinate channels

Location enters as **two extra input channels**: normalized latitude and longitude of every pixel in the crop, concatenated to the (noise-scaled) wind channels before the first convolution. This is the CoordConv idea (Liu et al., 2018 [7]) — give a translation-invariant CNN an explicit coordinate map when the task is *not* translation-invariant — and concatenating clean conditioning fields as input channels is the standard conditioning mechanism in image-to-image diffusion (SR3/Palette [8]) and, closest to home, in CorrDiff’s km-scale weather downscaling [9], where the coarse atmospheric state enters exactly this way.

Details that matter:

- **Normalization is part of the model contract.** Coordinates are mapped to $\approx[-1, 1]$ by the training domain’s center and half-width (lat $40^\circ \pm 15^\circ$, lon $240^\circ \pm 15^\circ$ for the NE Pacific box), and those four constants are stored in every checkpoint (`coord_norm`). Inference reproduces the exact mapping, including a guard that resolves the two longitude conventions ($-122.42^\circ \equiv 237.58^\circ$) onto the training branch.
- **Only the noisy target gets EDM’s `c_in` scaling; the coordinate channels enter clean** — they are already unit-scale and carry no noise, so scaling them with σ would corrupt a noiseless signal (same convention as the M3 route’s previous-frame conditioning).
- **Why per-pixel channels rather than a “crop is at (lato, lono)” scalar embedding?** Because it composes with tiling. When InfiniteDiffusion later tiles this denoiser across a large canvas, every window automatically receives *its own* coordinate map, and two overlapping windows agree about the coordinates of their shared pixels — so blending stays consistent and the generated canvas acquires true geographic structure (the jet sits at jet latitudes). A scalar embedding would give every tile a point-location and make within-window geographic gradients invisible.
- **Consequence: reflection augmentation had to go.** Mirroring a crop while keeping its coordinates teaches false geography; mirroring both trains on a mirrored Earth that never occurs at inference. The unconditional model could exploit the anomaly field’s reflection symmetry; a *located* model cannot. The full-year 2023 dataset ($\sim 6.5\times$ more real weather) replaces the $\times 4$ synthetic augmentation.

5. Conditioning pathway 2: time as cyclic harmonics (the deep dive)

5.1 The design question

“Condition on time” hides a real decision: *at what granularity?* Seasons (4 bins)? Months (12)? Day-of-year (365)? Every choice of discrete bin has three defects:

1. **Hard boundaries.** March 31 and April 1 have near-identical weather statistics but would receive different condition vectors; the model must waste capacity learning that adjacent bins are similar.
2. **No interpolation.** A balloon episode starting between bin centers gets a condition the model never saw varying — discrete embeddings don’t interpolate.
3. **The granularity is imposed, not learned.** Choosing “month” hard-codes the claim that sub-month variation doesn’t matter, before seeing any data.

And the opposite failure looms if the encoding is *too* fine: with a **single year** of training data, a model that can resolve individual days would learn “day-of-year 17” $\triangleq$ *the specific storm of Jan 17, 2023* — memorization of one year’s weather dressed up as seasonal conditioning.

5.2 The encoding

We encode each frame’s timestamp as **six numbers** (`data.time_features`):

```

 $\phi_{\text{year}} = 2\pi \cdot (\text{day of year}) / 365.25$ 
 $\phi_{\text{day}} = 2\pi \cdot (\text{UTC hour}) / 24$ 

t_feat = [ sin  $\phi_{\text{year}}$ ,  cos  $\phi_{\text{year}}$ ,      ← annual harmonic
           sin 2 $\phi_{\text{year}}$ , cos 2 $\phi_{\text{year}}$ ,    ← semiannual harmonic
           sin  $\phi_{\text{day}}$ ,   cos  $\phi_{\text{day}}$  ]    ← diurnal harmonic

```

Years are treated as **exchangeable** — only the phase within the year enters, never the year number. (Your framing: “the same month year to year would have the same weather” — this is the periodic-climate assumption, and with one training year we couldn’t learn inter-year variation anyway.)

5.3 Why sine and cosine — the circle argument

Time-of-year is a point on a circle, not a point on a line. A raw scalar (day 1...365) puts December 31 and January 1 maximally far apart when they are climatologically almost identical. The pair $(\sin \phi, \cos \phi)$ embeds the circle into the plane so that **Euclidean distance in feature space $\approx$ seasonal distance**: Dec 31 and Jan 1 are neighbors, June is maximally far from December, and the representation is continuous everywhere — no wrap seam. You need both components because sin alone is ambiguous ($\sin \phi = \sin(\pi - \phi)$: spring and autumn would collide); the pair identifies the phase uniquely. This is the same reason transformer positional encodings are sinusoid pairs (Vaswani et al., 2017 [10]).

5.4 Why harmonics — the bandlimit argument (the crux)

Here is the part that resolves the granularity question *structurally* rather than by picking a number. Any well-behaved periodic function of the annual phase can be written as a Fourier series:

$$f(\phi) = a_0 + a_1 \cos \phi + b_1 \sin \phi + a_2 \cos 2\phi + b_2 \sin 2\phi + a_3 \cos 3\phi + \dots$$

Feeding the network $\{\sin \phi, \cos \phi, \sin 2\phi, \cos 2\phi\}$ and nothing higher means every learned function of time-of-year is — at the input interface — a **band-limited** function: a smooth curve with at most two bumps per year. The network can build arbitrary nonlinear functions of *these four numbers*, but it can never construct a feature that distinguishes Jan 17 from Jan 20, because those two dates are nearly the same point in the input space. Consequences:

- **Granularity is learned, not imposed.** If the data only supports “winter vs summer,” the network learns slowly-varying weights on harmonic 1. If it supports sharper structure (e.g. a short monsoon-like transition), harmonic 2 gives it up to quarter-year resolution. Nothing forces either.
- **Single-year memorization is structurally impossible.** The failure mode of §5.1 — “day 17 means *that* storm” — would require high-frequency harmonics we simply don’t provide. Nearby dates share nearly identical embeddings, so the model can only learn what nearby dates have in *common*: the seasonal signal. This turns an overfitting worry into an architectural guarantee.
- **It scales with data.** If we later train on many years, adding harmonics 3 and 4 is a two-line change that *widens the band* — the encoding grows with what the data can support, rather than being redesigned.

Why exactly two harmonics for the year? It is the climatological standard: harmonic analysis of the seasonal cycle conventionally keeps the **annual + semiannual** pair, which captures the well-documented asymmetries of real seasonal cycles (seasons are not a pure sinusoid — e.g. lag of the temperature extremes, two-peaked tropical cycles); higher harmonics are mostly noise at climate scale (Wilks, *Statistical Methods in the Atmospheric Sciences*, ch. on harmonic analysis [11]). The diurnal cycle gets one harmonic; the semidiurnal atmospheric tide is real (especially in the stratosphere) but is deferred until a temporal metric demands it — one change at a time.

This is also what the strongest ML weather models feed their networks: **GraphCast** (Lam et al., 2023 [12]) inputs the *sine/cosine of year progress* and *local time of day* as per-grid-node forcings (§3.3: “the sine and cosine of the local time of day ... and the sine and cosine of the year progress”), and **GenCast** (Price et al., 2024 [13]) inherits the identical clock features (its Table A.1: “Local time of day”, “Elapsed year progress”). They use one harmonic per cycle; our encoding is their scheme plus the semiannual harmonic, which comes from the climatological convention above [11], not from them (and we feed UTC rather than local time — §5.5).

5.5 UTC, not local solar time

The diurnal phase uses **UTC hour**, not local solar hour. Local solar time is $\text{UTC} + \text{lon}/15\text{h}$ — a deterministic function of longitude, and the model *has* longitude (per-pixel, §4). So local time is learnable as a combination of two inputs it already receives, and feeding UTC keeps the time features independent of the coordinate channels instead of baking a redundant copy of longitude into them. (GraphCast chooses the opposite convention — local time — which is equivalent information; we prefer the factorized form since our longitude channel is per-pixel anyway.)

5.6 How the six numbers enter the network

The time features do **not** enter as image channels (they are spatially constant — as channels they’d waste convolution work). They ride the U-Net’s **embedding pathway**, the same side-channel that carries the noise level:

```
emb = MLP( FourierEmbedding(log  $\sigma$  / 4) )      # EDM's noise conditioning (per block)
emb = emb + Linear6( t_feat )                  # time conditioning (per frame!)
```

and `emb` is injected additively into every ResBlock (the `h + Linear(emb)` term inside each block). This “add your conditioning embedding to the timestep embedding” pattern is how class-conditional diffusion models condition (Dhariwal & Nichol, 2021 [14]) — the noise level σ and the calendar are treated as the same *kind* of information: global scalars that modulate every layer.

Two implementation details worth noticing:

- **Per-frame, not per-block.** Within a $\tau = 4$ block the hour advances frame to frame, so each frame gets its own `t_feat` row: the noise embedding (one σ per block) is broadcast across frames, and the per-frame time embedding is added on top. The model therefore knows not just “this block is a January night” but which frame is 02:00 vs 05:00 — the information needed to learn diurnal *evolution* inside a block.
- **Zero-initialized.** `Linear6` starts at exactly zero (weights and bias), so at initialization the conditional model is *identically* the unconditional one, and time sensitivity is learned only where it pays. Same philosophy as the TemporalConv (§3), same lineage [5, 6].

5.7 Putting it together: what the model can and cannot express

The conditioning interface hands the network, per frame: a smooth position on the annual circle (2 harmonics), a smooth position on the diurnal circle (1 harmonic), and a smooth map of where each pixel sits. Everything the model expresses about seasonality is some learned nonlinear function of those — so its seasonal behavior is guaranteed smooth in date, periodic across year boundaries, interpolable to any timestamp an RL episode requests, and incapable of encoding “that one storm.” What it *cannot* do, by design, is distinguish specific synoptic events at the same phase of the calendar — that variability must come from the diffusion noise, which is exactly where sample diversity belongs in a generative truth model. (If per-event control is ever wanted, the recorded fallback is CorrDiff-style coarse-field conditioning [9] — extra input channels, fully compatible with this architecture.)

6. Sampling (how a conditional block is generated)

```
SpaceTimeSampler.sample_block((H, W), seed, lat=..., lon=..., times=...):
```

1. Build the conditioning: coordinate channels from (lat, lon) via the checkpoint's stored `coord_norm`; `t_feat` rows from the τ timestamps.
2. Draw $x \sim N(0, \sigma_{\max}^2)$ of shape (τ, C, H, W) from the seed.
3. Run the 18-step Heun ODE, calling the denoiser with $(x, \sigma, \text{cond}, t_{\text{feat}})$ at every evaluation — the conditioning is constant along the trajectory; only x and σ evolve.
4. De-normalize per (level, variable) back to m/s.

Because the network is fully convolutional, H and W at sampling time need not equal the 64^2 training crop — the property the InfiniteDiffusion tiling relies on. The training gate (`gate.py`) evaluates checkpoints **condition-matched**: it samples the model at the same (lat, lon, times) tuples as real held-out training windows, so the comparison tests the conditional distribution, not just the pooled climate.

7. References

[1] Karras, Aittala, Aila, Laine. *Elucidating the Design Space of Diffusion-Based Generative Models* (EDM). NeurIPS 2022. arXiv:2206.00364. [2] Ho, Salimans, Gritsenko, Chan, Norouzi, Fleet. *Video Diffusion Models*. NeurIPS 2022. arXiv:2204.03458. [3] Tran, Wang, Torresani, Ray, LeCun, Paluri. *A Closer Look at Spatiotemporal Convolutions for Action Recognition* (R(2+1)D). CVPR 2018. arXiv:1711.11248. [4] Blattmann et al. *Align your Latents: High-Resolution Video Synthesis with Latent Diffusion Models*. CVPR 2023. arXiv:2304.08818. [5] Zhang, Rao, Agrawala. *Adding Conditional Control to Text-to-Image Diffusion Models* (ControlNet; zero-convolutions). ICCV 2023. arXiv:2302.05543. [6] Guo et al. *AnimateDiff: Animate Your Personalized Text-to-Image Diffusion Models* (zero-init temporal modules). ICLR 2024. arXiv:2307.04725. [7] Liu et al. *An Intriguing Failing of Convolutional Neural Networks and the CoordConv Solution*. NeurIPS 2018. arXiv:1807.03247. [8] Saharia et al. *Image Super-Resolution via Iterative Refinement* (SR3), arXiv:2104.07636; *Palette: Image-to-Image Diffusion Models*, arXiv:2111.05826 (conditioning by channel concatenation). [9] Mardani et al. *Residual Corrective Diffusion Modeling for Km-scale Atmospheric Downscaling* (CorrDiff). arXiv:2309.15214. [10] Vaswani et al. *Attention Is All You Need* (sinusoidal positional encoding). NeurIPS 2017. arXiv:1706.03762. [11] Wilks. *Statistical Methods in the Atmospheric Sciences* — harmonic analysis of the annual cycle (the annual + semiannual convention). [12] Lam et al. *Learning Skillful Medium-Range Global Weather Forecasting* (GraphCast; sin/cos of year progress + time of day as inputs). Science 2023. arXiv:2212.12794. [13] Price et al. *GenCast: Diffusion-based Ensemble Forecasting for Medium-Range Weather*. Nature 2024. arXiv:2312.15796. [14] Dhariwal, Nichol. *Diffusion Models Beat GANs on Image Synthesis* (class embedding added to the timestep embedding). NeurIPS 2021. arXiv:2105.05233.